FH-OÖ/Hagenberg/HSD
Chip-Design 3 — CHD3
Markus Pfaff, Daniel Windhager © 2026
Übung 2
.qsf, Know-what, DE23-Lite-Schaltung

MUX ist als Name für Multiplexer durch ein Bibliothekselement von IntelFPGA belegt. Leider bekommt man hierzu keine Fehlermeldung. Dieser Name sollte also vermieden werden. Besser wäre z.B. Mpx.

1. Aufgabe *Pimp your .qsf*

Im Verzeichnis `prjHelloOnDE23Lite\grpChd3\unitHelloOnDE23Lite\quartus` befindet sich das Quartus Settings File `HelloOnDE23Lite.qsf`, welches genauer untersucht werden soll. Öffnen Sie die Datei in ihrem Texteditor.

- Quartus hat einen eigenen Menüpunkt zur Restrukturierung dieser Datei: Project—Organize Quartus Prime Settings File. Führen Sie diese durch. Wie hat sich die Datei verändert?
- Am Anfang der Datei ist ein Hinweis von Altera eingetragen, der das Editieren der Datei betrifft. Was besagt dieser?
- Starten Sie den Compile-Prozess mit der veränderten Datei erneut. Quartus liest die Datei dabei neu ein. Nun öffnen Sie den *Assignment Editor* in Quartus.
- Dort sind für alle Pins des Designs zwei Einstellungen vorhanden, die nun wie in der QSF-Datei lesbarer sortiert sind. Die Einstellung *Location* ist spezifisch für jeden Pin, da damit der richtige Gehäuse-Pin dem jeweiligen *Port* der *entity* `HelloOnDE23Lite` zugeordnet wird. Beachten Sie, dass dies im Falle von Vektoren für deren Elemente im Einzelnen geschehen muss und nicht pauschal für den ganzen Vektor erledigt werden kann. Erläutern Sie kurz, warum das so ist.
- Die Einstellung für den Wert *I/O Standard* ist dagegen für alle Pins identisch. Dies kann man Quartus mit Hilfe sogenannter *Wildcards* (engl. für die Poker-Spielkarte Joker) für alle Pins in einer einzigen Zeile elegant mitteilen. Hierzu sollen zunächst alle Zeilen mit dem *Assignment I/O Standard* außer einer einzigen gelöscht werden: Markieren, rechte Maustaste, *Delete*. In der verbliebenen einzigen Zeile für *I/O Standard* in Spalte *To* doppelklicken und * eintragen. Dieses Zeichen steht für eine beliebige Anzahl beliebiger Zeichen, damit also jeden beliebigen Namen. Das Zeichen ? ist ebenfalls eine *Wildcard* und steht für *ein* beliebiges Zeichen.
- Nun im Compilation-Report unter *Fitter / Plan Stage / I/O Assignment Warnings* nachschauen. Hierzu gibt es auch eine entsprechende *Warning* von Quartus.
- Nun sollen die noch beanstandeten Einstellungen *drive strength und slew rate* (im *Assignment Editor* heißen sie: *Current Strength* und *Slew Rate*) für die richtigen Signale

hinzugefügt werden, wobei die Einstellungen *Minimum Current* respektive 0 verwendet werden können. Hierzu in der untersten Zeile im *Assignment Editor* (<<*new*>>) auf den *Assignment Name* doppelklicken und das gewünschte *Assignment* auswählen.

Beachten Sie, dass für jedes *Assignment* der Auswahl angegeben ist, ob *Wildcards* möglich sind. Nun in der neu angelegten Zeile den Eintrag für die Spalte *Value* doppelklicken und auswählen. Nun muss das Assignment noch vervollständigt werden, indem angegeben wird, worauf es überhaupt angewendet werden soll. Da es *Assignments* gibt, welche für Verbindungen zweier Punkte im FPGA gedacht sind, gibt es die beiden Spalten *From* und *To*. Für *Assignments*, welche sich auf einen einzigen Punkt innerhalb des FPGAs beziehen, ist die Spalte *From* ausgegraut.

Mit einem Doppelklick aktivieren Sie das Feld in der Spalte *To* nun für die Eingabe. Es erscheint ein Eingabefeld, ein Icon und ein Auswahlpfeil. Klicken Sie den Icon. Es öffnet sich das Fenster des *Node Finders*, der schon in der vorigen Übung kurz zu sehen war.

- Mittels *Node Finder* werden alle Ausgänge (nur um diese geht es) gesucht. Stellen Sie sicher dass nach dem Namen "*" gesucht wird und setzen Sie den Filter auf *Pins: all* um nur die Pins zu sehen.

  Gefunden werden unter anderem die Vektoren `HEX1`, `HEX2` und `LEDR`. Via Doppelklick kann nun der Pin Name direkt übernommen werden.

- Schauen Sie nun wieder in der QSF-Datei nach. Offenbar gibt es neben *Wildcards* noch eine zweite Möglichkeit, um mehrere *Assignments* in einer Zeile zu setzen. Und zwar?

- Prüfen Sie mit einem erneuten *Compile* in Quartus nach, ob die Warnungen, um die es ging nun nicht mehr vorhanden sind.

- Im *Compilation Report* unter Synthesis → Messages von Quartus sind noch immer einige *Warnings* zu finden:

  – `oLed(8)` wird nicht sinnvoll im VHDL-Code angesteuert.

  Gestalten Sie den VHDL-Code (und zugehörige *Assignments*) so um, dass diese *Warnings* beseitigt werden.

- Im *Message Window* von Quartus wird außerdem noch gewarnt über *High severity rule violations*. Es wird außerdem erwähnt in welchem File die nötigen Infos zu diese Violations zu finden sind. Über welche Violations warnt uns Quartus hier?

- Nun sollten Sie keinerlei Warnings mehr nach dem *Compile* in Quartus sehen – ein Zustand, der für jedes Design anzustreben ist.

2. Aufgabe *Fakultativ: Swell-Led*

Das Design Swell-LED ist ein Klassiker im Studiengang HSD. Es ist sogar zwei Monate älter als der Studiengang selbst. Mit zwei Tasten kann man eine LED in ihrer Helligkeit einstellen. Das Design ist im Archiv zu dieser Übung enthalten. Im Verzeichnis `doc` der *unit* `SwellLed` findet sich die genaue Spezifikation (Funktionsbeschreibung) des Designs.

Leider fehlen jedoch das *Testbed* und die QSF-Datei. Es ist lediglich eine QSF-Standarddatei von terasic vorhanden, welche *alle* Pins des FPGAs beschreibt und zusätzlich auch berücksichtigt, dass am FPGA SDRAM-Bausteine angeschlossen sind. Der überaus größte Teil dieser Datei ist also für diese Aufgabe irrelevant und kann gelöscht werden. Andere Einstellungen können unter Verwendung von *Wildcards* deutlich eleganter gemacht werden und wieder andere fehlen und sollten per *Assignment Editor* in Quartus später noch hinzugefügt werden.
Erstellen Sie ein Testbed (*entity* und *architecture* welches passende Tasten und LEDs des Boards an das Design anschließt. Berücksichtigen sie dabei Signale mit negativer Logik (z.B. `KEY`).
Achtung: Vergessen Sie nicht die Entity ResetReleaseIP in ihrem Testbed zu instanzieren, da ihr Design ansonsten nicht funktionieren wird!
Erstellen Sie ein Quartus-Projekt, führen Sie den *Compile*-Vorgang durch und spielen Sie den *Bitstream* auf dem Board ein. Nun sollten Sie die LED mittels der entsprechenden Tasten in der Helligkeit einstellen können.

3. Aufgabe *2zu1-Multiplexer*

In dieser Aufgabe geht es um einen 2zu1-Multiplexer (*entity* `Mpx`) mit den Eingängen *iA* und *iB*, welche auf den Ausgang *oY* gemultiplext werden. Der Auswahleingang *iSel* bestimmt, welcher der Eingänge auf den Ausgang *oY* durchgeschaltet wird.

Stellen Sie zunächst eine Wertetabelle des Multiplexers auf.

Der Multiplexer soll nun auf drei verschiedene Arten realisiert werden. Es werden nur die einfachen logischen Grundfunktionen **and**, **or**, ... verwendet. Die drei Beschreibungsarten sind:

- Mittels aller Minterme des *Karnaugh/Veitch-Diagramms*. Es existiert also ein Term pro Feld, das einen 1er enthält. *Architecture: Minterms*.
- Mittels der wesentlichen Primimplikanden. Es soll also zunächst minimiert werden. Danach wird das Minimierungsergebnis implementiert. *Architecture: PrimeImpl*.
- Ausschliesslich mit Hilfe der Funktion **nand**. *Architecture: NandOnly*.

Alle drei Implementationen sollen innerhalb der *entity* `AllMyMuxes` instanziiert werden. Dies dient zur Zusammenfassung der drei Multiplexer-Beschreibungen.
Für die Simulation wird diese *entity* mit einer *testbench* umgeben.
Um andererseits für die Synthese die Namen aus dem Schaltplan des Boards in der .qsf-Datei unverändert verwenden zu können, muss um die *entity* `AllMyMuxes` noch ein Platinenadapter gelegt werden. Insgesamt entsteht dann ein Gebilde nach Abb. 1, wobei die Top-Level *entitity* für die Synthese dann `AllMyMuxesOnDE23Lite` ist. Die Eingänge (z.B. von den Tastern) aller drei *entities* werden mit den gleichen *signals* beschaltet. Die Ausgänge werden natürlich getrennt betrachtet.

Um Ihre Beschreibungen lesbarer zu machen, verwenden Sie wo nötig bei positiver Logik die Konstanten `'0'` und `'1'` und bei negativer Logik `not('0')` und `not('1')` für den inaktiven, respektive aktiven Wert.
Nicht alle Signale kennen einen aktiven und einen inaktiven Wert. Ein Taktsignal wechselt beispielsweise einfach zwischen zwei Werten hin und her, von denen keiner als aktiv oder inaktiv betrachtet wird.

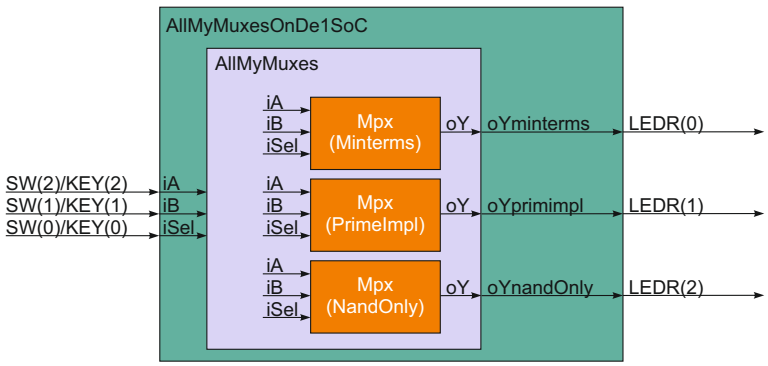


Abbildung 1: Schachtelung der Multiplexer in der *entity* `AllMyMuxes`, die wiederum von einem Platinenadapter umgeben ist.

Erstellen Sie für die *entity* `AllMyMuxes` eine *testbench* und weisen Sie durch Simulation nach, dass alle drei Entwürfe fehlerfrei funktionieren.

Um Ihren Entwurf auch auf dem Board ausprobieren zu können, erstellen Sie als Schale um ihren Entwurf einen Platinen- bzw. PCB-Adapter für unser Prototypenboard DE23-Lite namens `AllMyMuxesOnDE23Lite`. Belegen Sie die Eingänge mit den Schaltern (*iA: SW(2)*, *iB: SW(1)*, *iSel: SW(0)*) oder wahlweise mit den Tastern (*inA: KEY(2)*, *inB: KEY(1)*, *inSel: KEY(0)*) und die drei Ausgänge mit den LEDs *LEDR(0)*, *LEDR(1)* und *LEDR(2)*. Wenn Sie sicher sind, dass ihr Design und mehr noch die Zuordnung der FPGA-Pins zu den *Ports* aus der VHDL-Beschreibung fehlerfrei sind, probieren Sie den Entwurf auf dem Board aus.

Untersuchen Sie die entstehenden Schaltungen mit dem *RTL-Analyzer* (Tools → Netlist Viewers → RTL Analyzer).
Vergleichen Sie den Verbrauch an Ressourcen in den *Resource Usage Summarys* der Schritte *Synthesis* und *Fitter*. Gehen Sie dem Unterschied durch Vergleich der Schaltungen mit dem *Technology Map Viewer (Post-Mapping)* und dem *Technology Map Viewer (Post-Fitting)* auf den Grund.